

GOVERNMENT SCHOOL MANAGEMENT SYSTEM

Software Design Specification

SDS Document — System & Database Architecture

Document ID: GSMS-SDS-001

Version: 1.0

Date: 25 July 2026

Status: Draft for Review

1. Introduction

1.1 Purpose

This Software Design Specification (SDS) translates the requirements defined in the GSMS Software Requirements Specification (GSMS-SRS-001) into a concrete technical design: system architecture, database schema, module design, API contracts, security design and deployment architecture. It is intended for the development and QA teams as the primary technical reference during implementation.

1.2 Design Goals

- Modularity — each functional area (Student, Attendance, Examination, etc.) is a loosely coupled service/module behind a common API gateway layer.
- Security by default — every endpoint authenticated and authorized via JWT + RBAC; sensitive fields encrypted at rest where applicable.
- Extensibility — schema and API versioning designed to support future multi-school and Ministry-level rollout without breaking changes.
- Performance — indexed, normalized relational schema with caching at the API layer for read-heavy endpoints (timetables, reports).
- Maintainability — consistent layered structure (routes → controllers → services → data access) across all modules.

2. System Architecture

2.1 Architectural Style

GSMS follows a three-tier, layered client–server architecture, delivered as RESTful services consumed by a single-page application (SPA) frontend.

Layer	Technology	Responsibility
Presentation	React.js + Tailwind CSS	Role-based SPA UI, client-side routing, state management, form validation.
Application / API	Node.js + Express.js	REST API, business logic, authentication/authorization, orchestration across modules.
Data	PostgreSQL	Persistent relational storage, referential integrity, transactional consistency.
Cross-cutting	JWT + RBAC middleware	Authentication, session validation and permission enforcement on every request.

2.2 High-Level Architecture Diagram

The diagram below illustrates request flow from client through the API and authentication layer to the database and supporting external services.

Tier	Component	Connects To
Client	React SPA (Browser / Tablet)	API Gateway (HTTPS/REST)
Gateway	Express API Gateway + Auth Middleware (JWT/RBAC)	Module Controllers
Application	Student · Teacher · Staff · Academic · Attendance · Timetable · Examination · Library · Inventory · Welfare · Communication · Reports · User Mgmt controllers	Service Layer → Data Access Layer

Tier	Component	Connects To
Data	PostgreSQL (primary datastore)	Backup service, Reporting queries
External	Email/SMS gateway (Communication module)	Application layer (outbound only)

Client → HTTPS → API Gateway/Auth → Module Controllers → Service Layer → PostgreSQL, with the Communication module additionally invoking outbound email/SMS providers.

2.3 Module-to-Layer Mapping

Each of the thirteen functional modules identified in the SRS is implemented as a dedicated set of Express route + controller + service files sharing the common data-access and authentication middleware, following the pattern: **Route** (defines endpoint & applies RBAC guard) → **Controller** (request/response handling, validation) → **Service** (business logic) → **Repository/DAO** (parameterized SQL via a query builder/ORM) → PostgreSQL.

3. Database Design

3.1 Design Approach

The schema is normalized to 3NF to eliminate redundancy while keeping reporting queries efficient through targeted indexes and, where needed, materialized views for heavy aggregate reports. All tables use surrogate UUID primary keys, include created_at/updated_at audit timestamps, and reference a school_id foreign key to support future multi-school deployment.

3.2 Core Entity-Relationship Summary

Key relationships: a **School** has many Users, Students, Teachers, Staff and Classes. A **Student** belongs to one Class/Section per academic year and has many Guardians (many-to-many via a linking table), Attendance records, Exam Results and Welfare Enrolments. A **Teacher** is assigned to many Class-Subject pairs (many-to-many via Teaching_Assignment). A **Class** has one Timetable composed of many Timetable_Slots, each referencing a Subject and Teacher.

3.3 Key Table Schemas

users

Column	Type	Constraints
id	UUID	PK
school_id	UUID	FK → schools.id
email	VARCHAR(255)	UNIQUE, NOT NULL
password_hash	VARCHAR(255)	NOT NULL
role	ENUM	NOT NULL (principal, admin, teacher, staff, librarian, lab_assistant, student, parent, education_officer)
full_name	VARCHAR(150)	NOT NULL
is_active	BOOLEAN	DEFAULT true
created_at / updated_at	TIMESTAMP	NOT NULL

students

Column	Type	Constraints
id	UUID	PK
school_id	UUID	FK → schools.id
student_no	VARCHAR(30)	UNIQUE, NOT NULL
user_id	UUID	FK → users.id, NULLABLE
first_name / last_name	VARCHAR(100)	NOT NULL
date_of_birth	DATE	NOT NULL
class_id	UUID	FK → classes.id
admission_date	DATE	NOT NULL
status	ENUM	active, transferred, graduated, withdrawn

guardians / student_guardian

Column	Type	Constraints
id	UUID	PK (guardians)
full_name, phone, email	VARCHAR	guardians table
student_id	UUID	FK → students.id (link table)
guardian_id	UUID	FK → guardians.id (link table)
relationship	VARCHAR(30)	e.g. mother, father, guardian

teachers

Column	Type	Constraints
id	UUID	PK
user_id	UUID	FK → users.id
school_id	UUID	FK → schools.id
employee_no	VARCHAR(30)	UNIQUE
qualification	VARCHAR(255)	NULLABLE
subject_specialization	VARCHAR(150)	NULLABLE
max_periods_per_week	SMALLINT	DEFAULT 30

classes, subjects, teaching_assignments

Column	Type	Constraints
classes.id / grade / section / academic_year	UUID / VARCHAR / VARCHAR / SMALLINT	PK / NOT NULL
subjects.id / name / code	UUID / VARCHAR / VARCHAR	PK / UNIQUE code
teaching_assignments.class_id	UUID	FK → classes.id
teaching_assignments.subject_id	UUID	FK → subjects.id
teaching_assignments.teacher_id	UUID	FK → teachers.id

attendance

Column	Type	Constraints
id	UUID	PK
student_id / staff_id	UUID	FK, one of the two set per row
date	DATE	NOT NULL
status	ENUM	present, absent, late, excused
marked_by	UUID	FK → users.id
remarks	TEXT	NULLABLE

timetable_slots

Column	Type	Constraints
id	UUID	PK
class_id	UUID	FK → classes.id
subject_id / teacher_id	UUID	FK → subjects.id / teachers.id
day_of_week	SMALLINT	1–7
period_no / start_time / end_time	SMALLINT / TIME / TIME	NOT NULL
room	VARCHAR(30)	NULLABLE

exams, exam_results

Column	Type	Constraints
exams.id / name / term / academic_year	UUID / VARCHAR / VARCHAR / SMALLINT	PK / NOT NULL
exam_results.exam_id / student_id / subject_id	UUID	FK, composite unique
exam_results.marks_obtained / max_marks / grade	NUMERIC / NUMERIC / VARCHAR(5)	NOT NULL
exam_results.entered_by / entered_at	UUID / TIMESTAMP	audit trail

library_items, library_transactions

Column	Type	Constraints
library_items.id / title / author / category / copies_total / copies_available	UUID / VARCHAR... / SMALLINT	PK / NOT NULL
library_transactions.item_id / borrower_id	UUID	FK
library_transactions.issue_date / due_date / return_date / status	DATE / DATE / DATE / ENUM	issued, returned, overdue

inventory_items, inventory_transactions

Column	Type	Constraints
inventory_items.id / name / category / quantity / unit / reorder_level	UUID / VARCHAR... / NUMERIC / VARCHAR / NUMERIC	PK / NOT NULL
inventory_transactions.item_id / type / quantity / issued_to / date	UUID / ENUM / NUMERIC / UUID / DATE	type: receipt, issue, adjustment

welfare_programs, welfare_enrolments

Column	Type	Constraints
welfare_programs.id / name / eligibility_criteria / academic_year	UUID / VARCHAR / TEXT / SMALLINT	PK / NOT NULL
welfare_enrolments.program_id / student_id / status / disbursed_at	UUID / UUID / ENUM / TIMESTAMP	status: enrolled, disbursed, cancelled

announcements, notifications

Column	Type	Constraints
announcements.id / title / body / created_by / audience_role / audience_class_id	UUID / VARCHAR / TEXT / UUID / ENUM / UUID	PK / NOT NULL
notifications.id / recipient_id / channel / status / sent_at	UUID / UUID / ENUM / ENUM / TIMESTAMP	channel: in_app, email, sms

3.4 Indexing Strategy

- B-tree indexes on all foreign keys and on frequently filtered columns (attendance.date, exam_results.exam_id, students.class_id).
- Composite unique index on exam_results(exam_id, student_id, subject_id) to prevent duplicate marks entry.
- Partial index on notifications(status) where status = 'pending' to accelerate the delivery worker queue.

4. Module Design

Every module follows the same four-layer internal structure. The table below documents the primary responsibility of each layer, using the Attendance module as a representative example; all other modules (Student, Teacher, Staff, Academic, Timetable, Examination, Library, Inventory, Welfare, Communication, Reports, User Management) follow the identical pattern against their own tables.

Layer	Example File	Responsibility
Route	routes/attendance.routes.js	Defines /api/attendance endpoints; applies auth + RBAC middleware per route.
Controller	controllers/attendance.controller.js	Parses/validates request payloads; calls service; formats response.
Service	services/attendance.service.js	Business rules — e.g. absence-threshold checks, notification triggering.
Repository	repositories/attendance.repository.js	Parameterized SQL queries against PostgreSQL via the query builder.

4.1 Illustrative Sequence — Marking Attendance

- Teacher submits attendance for a class/period from the SPA (POST /api/attendance/bulk).
- API Gateway validates the JWT and confirms the teacher's RBAC permission for that class.
- Controller validates payload shape (student IDs belong to the specified class).
- Service layer persists attendance rows, then evaluates each student's rolling absence count.
- If a threshold is breached, the Service layer enqueues a notification via the Communication module's internal service interface.
- Repository commits the transaction; Controller returns a 201 response with a summary.

4.2 Inter-Module Interaction

Modules communicate internally through direct service-layer function calls within the same Node.js process (a modular monolith) rather than network calls, minimizing latency while the system serves a single school. The Communication module exposes an internal `notify()` service used by Attendance, Examination and Welfare to trigger parent/staff notifications. This boundary is designed so the Communication module can later be extracted into an independently scalable service without changing its calling contract.

5. API Design

All endpoints are versioned under `/api/v1`, return JSON, and require an Authorization: Bearer <JWT> header except for the login endpoint. Representative endpoints are listed below; the full contract is maintained in the OpenAPI specification delivered alongside the codebase.

Method	Endpoint	Description	Roles
POST	<code>/api/v1/auth/login</code>	Authenticate and issue JWT	All
POST	<code>/api/v1/students</code>	Register a student	Admin, Principal
GET	<code>/api/v1/students/{id}</code>	Retrieve student profile	Admin, Teacher, Parent (own), Student (own)
POST	<code>/api/v1/attendance/bulk</code>	Submit period attendance	Teacher, Admin
GET	<code>/api/v1/timetable/class/{classId}</code>	Get published timetable	All (school-scoped)
POST	<code>/api/v1/exams/{examId}/results</code>	Submit marks	Teacher
GET	<code>/api/v1/reports/attendance-summary</code>	Attendance report	Admin, Principal, Education Officer
POST	<code>/api/v1/library/transactions/issue</code>	Issue a library item	Librarian
POST	<code>/api/v1/welfare/{programId}/enrol</code>	Enrol student in welfare program	Admin
POST	<code>/api/v1/announcements</code>	Broadcast announcement	Admin, Teacher, Principal

Standard error responses follow a consistent envelope: `{ "error": { "code", "message", "details" } }`, with HTTP status codes used semantically (400 validation, 401 unauthenticated, 403 unauthorized, 404 not found, 409 conflict, 500 server error).

6. Security Design

Control	Implementation
Authentication	JWT (short-lived access token + refresh token), signed with a rotating secret; refresh tokens stored server-side and revocable.
Authorization	RBAC middleware evaluated per-route and, where needed, per-record (e.g. a parent may only view their own linked children).
Password storage	bcrypt/argon2 hashing with per-user salt; no plaintext or reversible storage.
Transport security	TLS 1.2+ enforced for all traffic; HSTS enabled.
Input validation	Schema-based request validation (e.g. Joi/Zod) on every endpoint; parameterized queries to prevent SQL injection.
Audit logging	Append-only <code>audit_log</code> table capturing actor, action, entity, before/after diff and timestamp for sensitive operations.
Rate limiting	IP- and account-based rate limiting on authentication endpoints to mitigate brute force.
Data at rest	Database-level encryption for storage volumes; column-level encryption considered for highly sensitive fields in future iterations.

7. UI Design Guidelines

The frontend uses a consistent design language across all role dashboards to reduce training overhead for less technical users (Principals, Librarians, Parents).

- Layout: persistent left navigation grouped by module, top bar with role/school context and notifications.
- Responsiveness: mobile-first Tailwind breakpoints; core workflows (attendance, results viewing) usable on a phone-sized screen.
- Forms: inline validation with clear error messaging; destructive actions (delete, de-register) require confirmation dialogs.
- Data tables: paginated, sortable, filterable; exportable to PDF/Excel per the Reports requirements.
- Accessibility: WCAG 2.1 AA color contrast, full keyboard navigability, semantic HTML landmarks.

8. Deployment Architecture

The application is containerized (Docker) for consistent deployment across environments. A typical production topology consists of:

- A reverse proxy / load balancer (e.g. Nginx) terminating TLS and routing to API containers.
- One or more stateless Node.js/Express API containers, horizontally scalable.
- A managed or self-hosted PostgreSQL instance with automated daily backups and point-in-time recovery.
- A static asset host/CDN serving the built React SPA.
- Separate environments for Development, Staging (UAT) and Production, with environment-specific configuration via environment variables/secrets management.

8.1 Environments

Environment	Purpose	Data
Development	Active feature development	Synthetic/seed data only
Staging / UAT	User Acceptance Testing before release, per the Development Methodology document	Anonymized or sanitized sample data
Production	Live school operation	Real student/staff data, access-restricted